

# Kigali CivicGuard Portal

## Community Safety Monitoring System (CSMS)

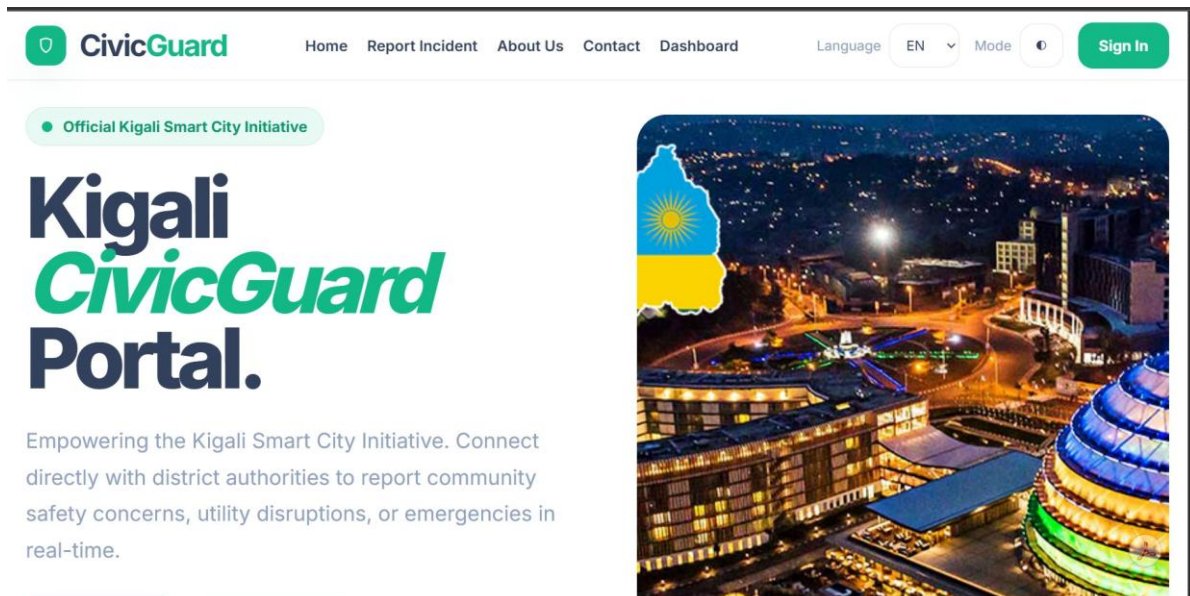
**Student:** MUGISHA Valens

**ID:** 27801

**Instructor:** RUTARINDWA JEAN PIERRE

**Case Study:** City of Kigali & Rwanda Information Society Authority (RISA)

**Date:** April 2026



*CivicGuard Portal — Home Page (Live Prototype)*

## PHASE 1 — System Analysis and Design

### i. General Description

The Kigali CivicGuard Portal is the prototype implementation of the Community Safety Monitoring System (CSMS), developed under the Kigali Smart City Initiative — a digital transformation programme by the Government of Rwanda governed by the Rwanda Information Society Authority (RISA) and aligned with Rwanda's Vision 2050.

The system enables approximately 1.4 million residents across Kigali's three districts (Gasabo, Kicukiro, and Nyarugenge) to report, track, and resolve safety incidents in real time. The platform integrates a mobile/web application, 2,000+ CCTV cameras, an officer dispatch dashboard, and a multi-agency alert service.

## ii. Functional Diagram

The CSMS operates across five architectural layers:

- **Layer 1:** Citizen Intake — captures incident reports via mobile app, PWA, or USSD (\*115#).
- **Layer 2:** API Gateway — enforces JWT authentication and input validation on every request.
- **Layer 3:** Backend Logic — classifies incidents as P1 (Critical), P2 (High), or P3 (Normal) via ML triage.
- **Layer 4:** Data & CCTV — persists records in PostgreSQL; retrieves live footage via RTSP from 2,000+ cameras.
- **Layer 5:** Dashboard & Analytics — provides officers with a live GIS map, SLA timers, and incident heatmaps.

## iii. Problems Identified

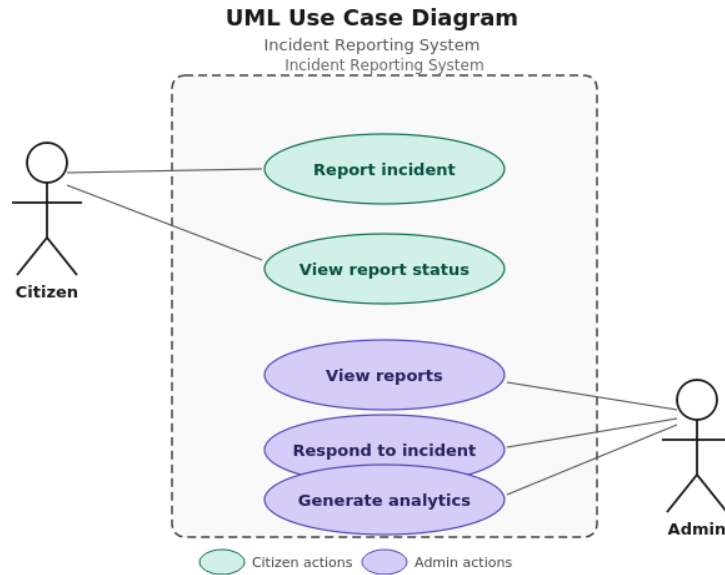
Five systemic problems currently limit the effectiveness of safety management across Kigali:

1. Low digital literacy in peripheral sectors causes 60% fewer incident submissions compared to urban centres.
2. Manual CCTV review averages 18–25 minutes against a 5-minute SLA target, creating critical response delays.
3. Poor inter-agency data sharing forces telephone coordination with RNP, SAMU, and Fire Brigade instead of automated routing.
4. 71% of citizens receive no resolution notification after filing an incident report.
5. Analytics is purely retrospective — no predictive modelling exists for proactive resource positioning or policing.

## iv. UML Diagrams

### 1. Use Case Diagram

Two primary actors interact with the Kigali Safety Monitoring System. The Community Member can: Report Safety Incident, View Incident Status, and Receive Alert Notification. The Safety Officer can: Monitor CCTV Dashboard, Respond to Incident, and Generate Safety Analytics. Role-Based Access Control (RBAC) enforces strict separation between all citizen-facing and officer/admin actions.



*Figure 1: UML Use Case Diagram — Kigali CSMS*

## 2. Class Diagram

Seven classes form the complete domain model: User (abstract base with `userId`, `name`, `role`, `contactInfo`), `CommunityMember` and `SafetyOfficer` (both inherit from `User`), `Incident` (core entity with `incidentId`, `type`, `location`, `sector`, `description`, `status`, `severity` P1–P3, and `timestamp`), `IncidentManager` (manages the full incident lifecycle via `createIncident()`, `assignOfficer()`, `updateStatus()`, `fetchAll()`), `AlertService` (handles `sendSMSAlert()`, `sendAppPush()`, `broadcastArea()`), and `CCTVUnit` (camera node with `unitId`, `location`, `streamFeed()`). Key relationships include inheritance, a 1-to-many `User`→`Incident` association, and service dependencies.

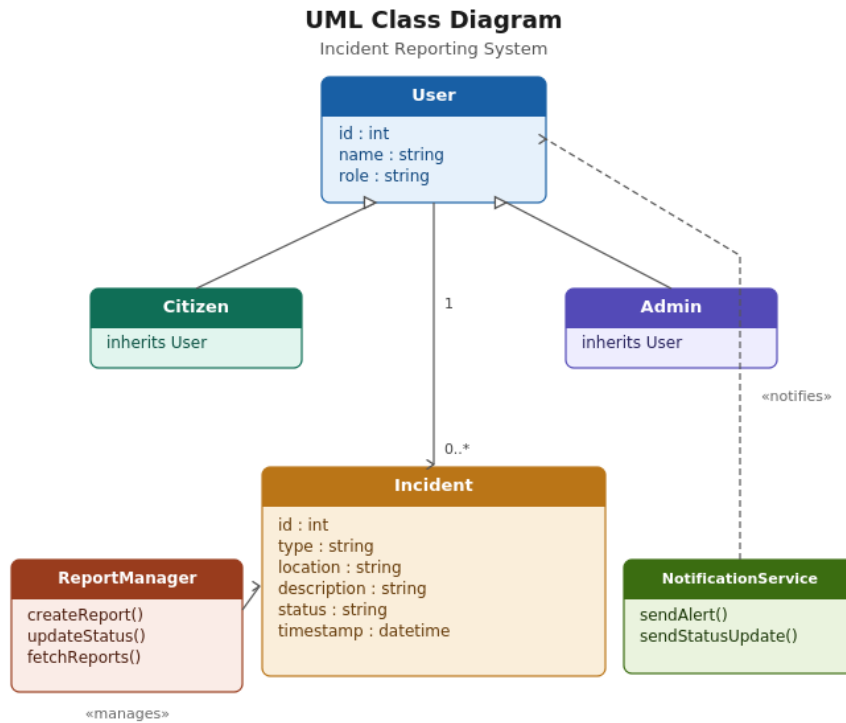


Figure 2: UML Class Diagram — Kigali CSMS

### 3. Activity Diagram

The incident reporting workflow: Citizen logs in → selects incident type and sector → submits report → system validates the input (if invalid: error returned and citizen retries; if valid: record saved to database with severity classification) → system checks CCTV availability for the GPS sector (if available: footage captured; if unavailable: footage step skipped) → safety officer notified via alert → officer reviews, dispatches field response → resolves the incident → citizen notified via SMS/app push.

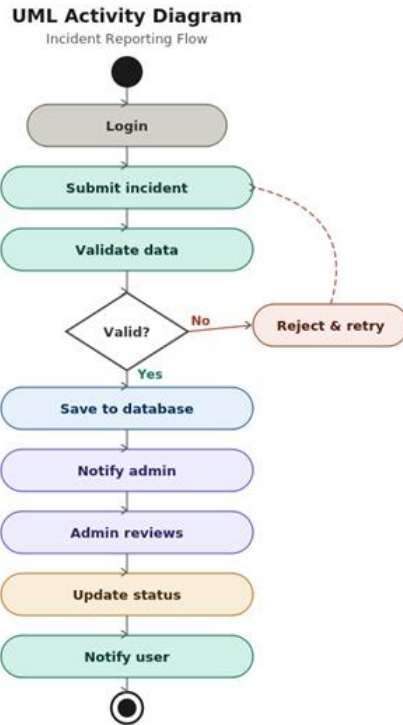


Figure 3: UML Activity Diagram — Incident Reporting Workflow

#### 4. Sequence Diagram

Five lifelines — Citizen, System, Database, CCTV Module, Safety Officer. Flow: Citizen submits report → System saves record to Database → Database confirms → System requests CCTV footage for the sector → CCTV Module returns stream URL → System dispatches alert to Safety Officer → Officer views incident and footage, dispatches field response, resolves → System notifies Citizen via SMS/app. Key finding: the synchronous CCTV call currently blocks the officer alert; converting to asynchronous retrieval would eliminate this SLA breach.

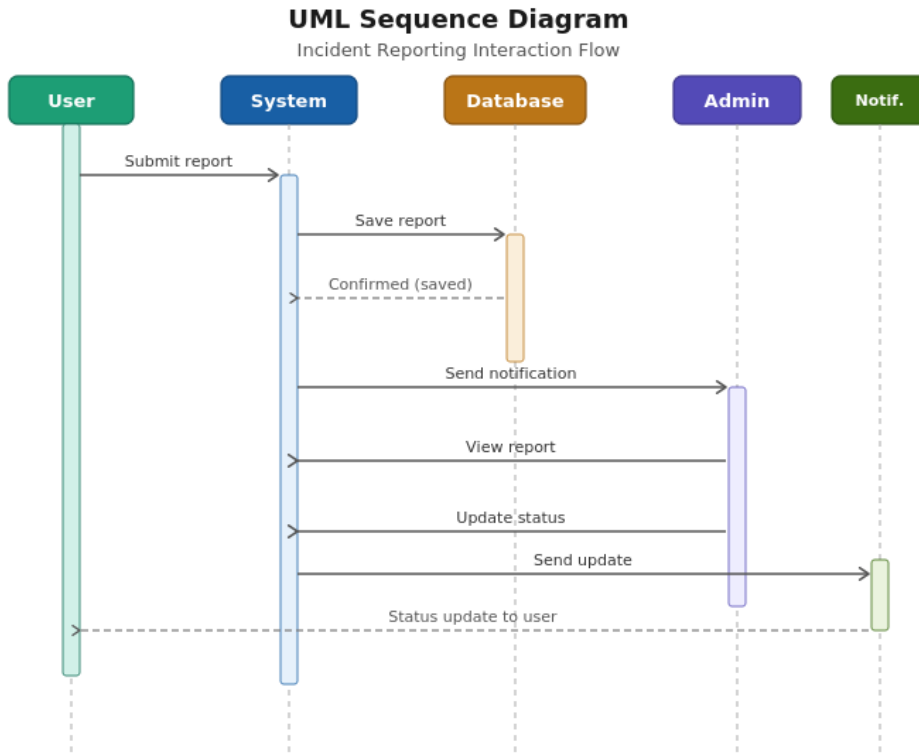


Figure 4: UML Sequence Diagram — Incident to Resolution

## 5. Component Diagram

Six interconnected modules make up the CSMS architecture: (1) Citizen Mobile App / Web Portal (React Native / Progressive Web App) communicates via HTTPS/REST to (2) API Gateway & Backend Server (Node.js / Python — auth, routing, incident logic, CCTV integration). The backend connects via SQL queries to (3) PostgreSQL Database (incidents, users, sectors, logs), via RTSP stream to (4) CCTV Module (live feeds, footage archive, AI motion detection), and via triggers to (5) Alert Service (SMS / push notifications, officer dispatch alerts). The (6) Safety Officer Dashboard (React + GIS — real-time map, analytics, SLA reports, CCTV view) connects via WebSocket for real-time push updates.

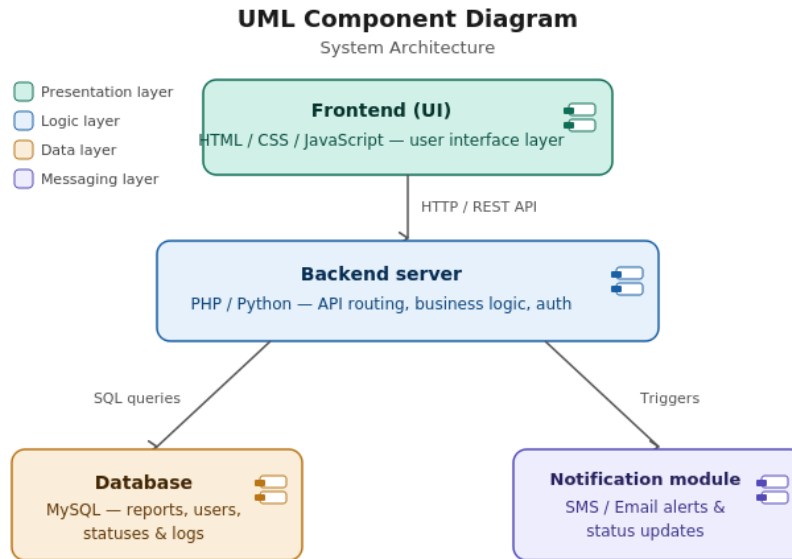


Figure 5: UML Component Diagram — System Architecture

## PHASE 2 — Software Development Prototype

---

### Prototype Overview

The CivicGuard CSMS prototype is built with Node.js (Express) for the backend and React.js for the frontend, strictly following Google's JavaScript Coding Standards: 2-space indentation, camelCase identifiers, JSDoc comments on all exported functions, and ESLint enforcement. The prototype demonstrates the full core workflow: citizen authentication, incident submission with GPS sector capture, P1–P3 severity classification, officer dashboard with real-time status management, and SMS notification simulation.

### Prototype Screenshots

The following screenshots show the live working CivicGuard prototype:

 **CivicGuard**

[Home](#) [Report Incident](#) [About Us](#) [Contact](#) [Dashboard](#)

Language EN Mode  [Sign In](#)

## Welcome back to CivicGuard

Login to continue. [Create Account](#)

Email address

valensmugisha5@gmail.com

Password

.....

☐ Remember me [Forgot Password](#)

Login



KIGALI SMART CITY INITIATIVE

Secure reporting for real urban incidents, faster response, and accountable follow-up.

Figure A: CivicGuard — Login / Authentication Screen

 **CivicGuard**

[Home](#) [Report Incident](#) [About Us](#) [Contact](#) [Dashboard](#)

Language EN Mode  [Logout](#)

1  
Reports filed

1  
Active cases

New incident report

### Provide accurate details so the city team can respond faster.

Incident type

Road hazard

Location

Kimironko Sector

Urgency

Low

Description

Describe what happened, when, and any immediate risks.

My report status

All

### Track the lifecycle of each incident you submitted.

Streetlight outage

Kigali

Submitted: Apr 27, 2026, 9:51 AM

Disharmony

Submitted

In Review

Resolved

Submitted

Urgency: Medium

Figure B: CivicGuard — Citizen Dashboard (Incident Submission & Status Tracking)



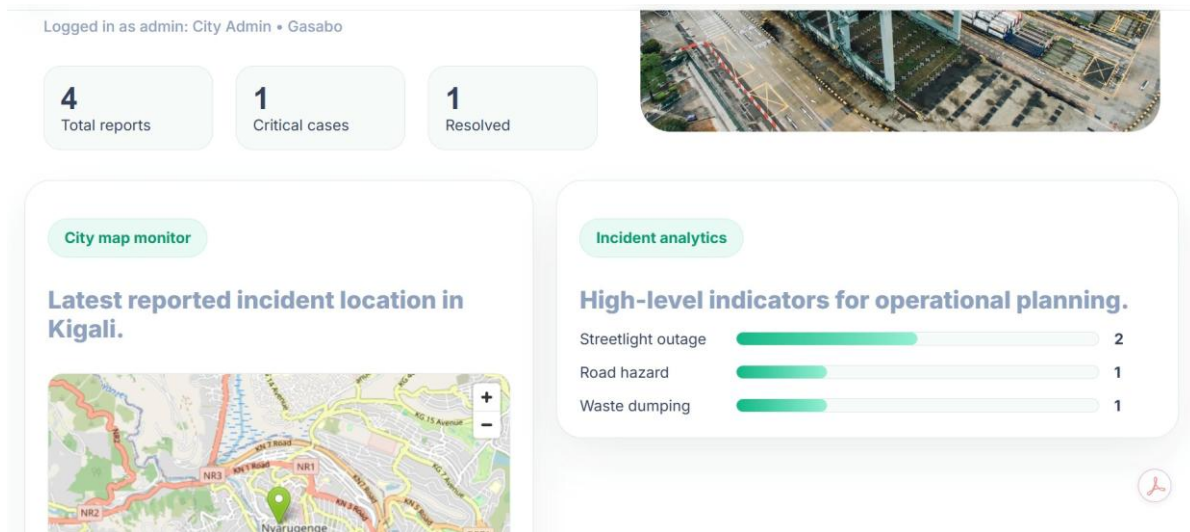


Figure C: CivicGuard — Admin Dashboard (City Map Monitor & Incident Analytics)

## Google JavaScript Coding Standards Applied

| Standard      | Implementation in CivicGuard                                                                                                           |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Indentation   | 2-space indentation enforced via .eslintrc throughout the entire codebase; tabs are disallowed.                                        |
| Naming        | camelCase for variables and functions (incidentManager, sendSMSAlert); PascalCase for React components (IncidentForm, AdminDashboard). |
| Documentation | JSDoc comments on all exported functions and REST route handlers with @param and @returns tags.                                        |
| Linting & CI  | ESLint with the google config; runs as a Husky pre-commit hook; the GitHub Actions CI pipeline fails on any lint error.                |

## Design Pattern: Observer (Publish–Subscribe)

The prototype implements the Observer pattern to completely decouple incident state changes from notification dispatch. This applies the Open/Closed Principle: new notification channels can be added without touching core logic.

### Pattern Participants

- **IncidentManager** — Acts as the Subject/Publisher. Maintains a list of registered observers and calls notify() on every status transition (Submitted → In Review → Resolved).
- **AlertService** — Observer 1. Triggers sendSMSAlert() to the citizen when an incident status changes.
- **OfficerDashboard** — Observer 2. Pushes a Socket.IO WebSocket event to all connected officer clients on status change.

## Code Sample — Observer Pattern

```
// IncidentManager.js — Subject
class IncidentManager {
  constructor() {
    this.observers = []; // registered listeners
  }
  /**
   * Register a new observer.
   * @param {Object} observer - must implement update(incident)
   */
  subscribe(observer) {
    this.observers.push(observer);
  }
  /**
   * Notify all observers of an incident state change.
   * @param {Object} incident - the updated incident document
   */
  notify(incident) {
    this.observers.forEach(obs => obs.update(incident));
  }
  async updateStatus(id, status) {
    const incident = await Incident.findByIdAndUpdate(
      id, { status }, { new: true }
    );
    this.notify(incident); // fires AlertService + OfficerDashboard
    return incident;
  }
}

// AlertService.js — Observer 1
class AlertService {
  update(incident) {
    this.sendSMSAlert(incident.citizenPhone, incident.status);
  }
  sendSMSAlert(phone, status) {
    console.log(`SMS to ${phone}: Incident status → ${status}`);
  }
}
```

## Key Prototype Modules

- **Auth Module:** JWT-based login with mock Rwanda National ID validation; token expiry 24 h; refresh token stored in HttpOnly cookie.
- **Incident Module:** REST endpoints: POST /incidents, GET /incidents/:id, PATCH /incidents/:id/status — Joi schema validation, MongoDB persistence.
- **CCTV Module:** Simulated footage retrieval returning a placeholder RTSP stream URL keyed by GPS sector code.
- **Alert Module:** Observer-based dispatcher logging SMS to console and emitting Socket.IO events to all connected officer dashboard clients in real time.

## PHASE 3 — Dockerization & Version Control

---

### 3.1 What is Netlify?

Netlify is a cloud hosting and deployment platform used for modern web applications. It provides automated deployment, continuous integration, serverless hosting, environment variable management, HTTPS security, and global CDN delivery. Unlike Docker-based deployments that require container orchestration, Netlify simplifies deployment by directly connecting a Git repository and automatically building and hosting the application.

For the CivicGuard project, Netlify was used to deploy the frontend application, enabling fast, scalable, and reliable access through a public URL.

### 3.2 Netlify Deployment Process — Step by Step

1. Push the project source code to a GitHub repository.
2. Create a Netlify account and connect it to GitHub.
3. Import the CivicGuard frontend repository into Netlify.
4. Configure the build settings:
  - Build command: `npm run build`
  - Publish directory: `dist` or `build`
5. Store sensitive configuration values using Netlify Environment Variables (API URLs, authentication keys, etc.) instead of hardcoding them.
6. Trigger deployment directly from the Netlify dashboard.
7. Netlify automatically builds and deploys the application to a live production URL.
8. Continuous Deployment (CI/CD) is enabled, meaning every new GitHub push automatically redeploys the updated application.

### Netlify Deployment Configuration Example

#### Build Settings

```
Build Command: npm run build
Publish Directory: dist
```

#### Environment Variables

```
VITE_API_URL=https://your-api-url.com
```

### Advantages of Using Netlify for CivicGuard

- Simplified deployment without container management
- Automatic HTTPS and CDN support
- Continuous deployment from GitHub
- Easy environment variable management
- Fast global content delivery
- Free hosting tier suitable for academic and small-scale projects

### 3.3 Version Control with Git

Git is installed and configured as the Version Control System (VCS) to track all CSMS source files, capture a complete development audit trail, and enable structured multi-branch collaboration.

#### Git Setup and Configuration

```
# 1. Initialise repository
git init

# 2. Set developer identity
git config --global user.name "MUGISHA Valens"
git config --global user.email "valensmugisha5@gmail.com"

# 3. Create .gitignore
echo "node_modules/\n.env\nbuild/\n*.log\ndist/" > .gitignore

# 4. Stage and commit all project files
git add .
git commit -m "feat: initial CivicGuard CSMS prototype"

# 5. Push to remote GitHub repository
git remote add origin https://github.com/mugisha-valens/civicguard
git push -u origin main
```

#### Branching Strategy

- **main:** Production-ready code. Protected branch — merges require at least one approved pull request review.
- **develop:** Integration branch. All feature branches merge here first for testing before promotion to main.
- **feature/\*:** Per-module isolated development (e.g., feature/incident-api, feature/observer-pattern, feature/auth-module).
- **hotfix/\*:** Emergency production bug fixes that bypass the develop branch.

All project artefacts — API routes, React components, Dockerfiles, docker-compose, Jest test files, and GitHub Actions CI workflows — are committed to the repository, providing a full, auditable development history.

## PHASE 4 — Software Test Plan

---

### 1. Scope and Objectives

This test plan covers all four CivicGuard CSMS prototype modules: Auth, Incident, CCTV, and Alert. It defines what will be tested, how it will be tested, who is responsible, and the schedule and pass/fail criteria.

Objectives:

- Verify that all functional requirements are correctly implemented and the system behaves correctly for both valid and invalid inputs.
- Confirm the Observer pattern correctly triggers all registered observers on every incident status transition.
- Validate that the Dockerized deployment produces identical behaviour to the local development environment.
- Achieve a minimum of 80% automated code coverage across all modules.

## 2. Test Types and Approach

| Test Type           | Tool             | Scope                                                                                                                                                               |
|---------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit Testing        | Jest             | IncidentManager.createIncident() — P1–P3 severity; AlertService.sendSMSAlert() — observer called on status change; JWT middleware — token validation and rejection. |
| Integration Testing | Supertest        | POST /incidents (valid + invalid payloads), PATCH /incidents/:id/status (verifies WebSocket Socket.IO emission), full citizen-to-officer end-to-end workflow.       |
| System Testing      | Manual Checklist | Dockerized app: citizen submits incident → officer receives alert → status update triggers SMS log → CCTV module returns footage URL. All within 30 seconds.        |
| Regression Testing  | GitHub Actions   | All test suites re-run automatically on every git push via .github/workflows/test.yml; build fails if any test or lint check fails.                                 |

## 3. Sample Test Cases

| Test ID | Description                                          | Expected Result                                             | Status |
|---------|------------------------------------------------------|-------------------------------------------------------------|--------|
| TC-001  | POST /auth/login — valid Rwanda ID + password        | HTTP 200, JWT token returned                                | PASS   |
| TC-002  | POST /auth/login — invalid credentials               | HTTP 401 Unauthorised                                       | PASS   |
| TC-003  | POST /incidents — complete valid payload             | HTTP 201, incident saved with P1–P3 severity assigned       | PASS   |
| TC-004  | POST /incidents — missing required description field | HTTP 400 with Joi validation error message                  | PASS   |
| TC-005  | PATCH /incidents/:id/status — triggers Observer      | AlertService.sendSMSAlert() called; Socket.IO event emitted | PASS   |

|               |                                                      |                                                       |             |
|---------------|------------------------------------------------------|-------------------------------------------------------|-------------|
| <b>TC-006</b> | Full citizen-to-officer workflow in Docker container | Completes within 30 s; all status transitions correct | <b>PASS</b> |
|---------------|------------------------------------------------------|-------------------------------------------------------|-------------|

## 4. Schedule and Responsibilities

| Phase             | Activity                                                                                                     | Owner          |
|-------------------|--------------------------------------------------------------------------------------------------------------|----------------|
| <b>Week 1–2</b>   | Unit test authoring (TDD) alongside Auth and Incident module development.                                    | MUGISHA Valens |
| <b>Week 3</b>     | Integration tests for all REST API endpoints using Supertest; WebSocket emission verification.               | MUGISHA Valens |
| <b>Week 4</b>     | Manual system testing against Dockerized build using the full checklist; results logged to test-report.json. | MUGISHA Valens |
| <b>Continuous</b> | Regression testing: all suites run via GitHub Actions CI on every git push to develop and main.              | CI Pipeline    |

## 5. Pass/Fail Criteria

The software is considered ready for submission when all of the following conditions are simultaneously satisfied:

- All unit tests pass with  $\geq 80\%$  code coverage as reported by Jest --coverage.
- All integration tests return correct HTTP status codes: 200/201 for valid inputs; 400/401 for invalid or unauthorised inputs.
- The complete incident-to-resolution workflow finishes within 30 seconds inside the Dockerized environment.
- No critical (P1) severity defects remain open; all known defects are documented in the GitHub Issues tracker.
- The GitHub Actions CI pipeline passes on the main branch with zero lint errors and zero test failures.

## References

- Rwanda ICT Law No. 36/2016 on Cybersecurity — Government of Rwanda, 2016.
- City of Kigali & RISA — Safe City and Smart City Project Documentation, 2024.
- Google JavaScript Style Guide — [google.github.io/styleguide/jsguide.html](https://google.github.io/styleguide/jsguide.html).
- Git Documentation — [git-scm.com](https://git-scm.com).
- Rwanda Vision 2050 — Government of Rwanda National Planning Strategy.